c/c++测试函数的运行时间（八种方法）（转）

转载 jhcconan614 最后发布于 2017-02-22 20:04:32 阅读数 20954 收藏

展开

目前，存在着各种计时函数，一般的处理都是先调用计时函数，记下当前时间 tstart，然后处理一段程序，再调用计时函数，记下处理后的时间 tend，再 tend 和 tstart 做差，就可以得到程序的执行时间，但是各种计时函数的精度不一样.下面对各种计时函数，做些简单记录.

```
void foo()
{
    long i;
    for (i=0;i<100000000;i++)
    {
        long a= 0;
        a = a+1;
    }
}
```

方法 1,time()获取当前的系统时间，返回的结果是一个 time_t 类型，其实就是一个大整数，其值表示从 CUT（Coordinated Universal Time）时间 1970 年 1 月 1 日 00:00:00（称为 UNIX 系统的 Epoch 时间）到当前时刻的秒数.

```
void test1()
{
    time_t start,stop;
```

```
    start = time(NULL);
    foo();//dosomething
    stop = time(NULL);
    printf("Use Time:%ld\n",(stop-start));
}
```

方法 2,clock()函数返回从“开启这个程序进程”到“程序中调用 clock()函数”时之间的 CPU 时钟计时单元（clock tick）数，在 MSDN 中称之为挂钟时间（wal-clock）常量 CLOCKS_PER_SEC，它用来表示一秒钟会有多少个时钟计时单元。

```
void test2()
{
    double dur;
    clock_t start,end;
    start = clock();
    foo();//dosomething
    end = clock();
    dur = (double)(end - start);
    printf("Use Time:%f\n",(dur/CLOCKS_PER_SEC));
}
```

方法 3,timeGetTime()函数以毫秒计的系统时间。该时间为从系统开启算起所经过的时间,是 windows api

```
void test3()
{
    DWORD t1,t2;
    t1 = timeGetTime();
    foo();//dosomething
    t2 = timeGetTime();
    printf("Use Time:%f\n",(t2-t1)*1.0/1000);
```

```
}
```

方法 4,QueryPerformanceCounter()这个函数返回高精确度性能计数器的值,它可以以微妙为单位计时.但是 QueryPerformanceCounter()确切的精确计时的最小单位是与系统有关的,所以,必须要查询系统以得到 QueryPerformanceCounter()返回的嘀哒声的频率.QueryPerformanceFrequency()提供了这个频率值,返回每秒嘀哒声的个数.

```
void test4()
{
	LARGE_INTEGER t1,t2,tc;
	QueryPerformanceFrequency(&tc);
	QueryPerformanceCounter(&t1);
	foo();//dosomething
	QueryPerformanceCounter(&t2);
	printf("Use Time:%f\n",(t2.QuadPart - t1.QuadPart)*1.0/tc.QuadPart);
}
```

方法 5,GetTickCount 返回（retrieve）从操作系统启动到现在所经过（elapsed）的毫秒数，它的返回值是 DWORD

```
void test5()
{
	DWORD t1,t2;
	t1 = GetTickCount();
	foo();//dosomething
	t2 = GetTickCount();
	printf("Use Time:%f\n",(t2-t1)*1.0/1000);
}
```

方法 6,RDTSC 指令，在 Intel Pentium 以上级别的 CPU 中，有一个称为“时间戳（Time Stamp）”的部件，它以 64 位无符号整型数的格式，记录了自 CPU 上电以来所经过的时钟周期数。由于目前的 CPU 主频都非常高，因此这

个部件可以达到纳秒级的计时精度。这个精确性是上述几种方法所无法比拟的.

在 Pentium 以上的 CPU 中，提供了一条机器指令 RDTSC（Read Time Stamp Counter）来读取这个时间戳的数字，并将其保存在 EDX:EAX 寄存器对中。由于 EDX:EAX 寄存器对恰好是 Win32 平台下 C++语言保存函数返回值的寄存器，所以我们可以把这条指令看成是一个普通的函数调用，因为 RDTSC 不被 C++的内嵌汇编器直接支持，所以我们要用_emit 伪指令直接嵌入该指令的机器码形式 0X0F、0X31

```
inline unsigned __int64 GetCycleCount()
{
	__asm
	{
		_emit 0x0F;
		_emit 0x31;
	}
}

void test6()
{
	unsigned long t1,t2;
	t1 = (unsigned long)GetCycleCount();
	foo();//dosomething
	t2 = (unsigned long)GetCycleCount();
	printf("Use Time:%f\n",(t2 - t1)*1.0/FREQUENCY);	//FREQUENCY 指 CPU 的频率
}
```

方法 7,gettimeofday() linux 环境下的计时函数，int gettimeofday (struct timeval * tv , struct timezone * tz),gettimeofday()会把目前的时间有 tv 所指的结构返回，当地时区的信息则放到 tz 所指的结构中.

```
//timeval 结构定义为:
struct timeval{
long tv_sec; /*秒*/
long tv_usec; /*微秒*/
};
//timezone 结构定义为:
struct timezone{
int tz_minuteswest; /*和 Greenwich 时间差了多少分钟*/
int tz_dsttime; /*日光节约时间的状态*/
};
void test7()
{
    struct timeval t1,t2;
    double timeuse;
    gettimeofday(&t1,NULL);
    foo();
    gettimeofday(&t2,NULL);
    timeuse = t2.tv_sec - t1.tv_sec + (t2.tv_usec - t1.tv_usec)/1000000.0;
    printf("Use Time:%f\n",timeuse);
}
```

方法 8,linux 环境下，用 RDTSC 指令计时.与方法 6 是一样的.只不过在 linux 实现方式有点差异.

```
#if defined (__i386__)
static __inline__ unsigned long long GetCycleCount(void)
{
	unsigned long long int x;
	__asm__ volatile("rdtsc":"=A"(x));
	return x;
}
#elif defined (__x86_64__)
static __inline__ unsigned long long GetCycleCount(void)
{
	unsigned hi,lo;
	__asm__ volatile("rdtsc":"=a"(lo),"=d"(hi));
	return ((unsigned long long)lo)|(((unsigned long long)hi)<<32);
}
#endif

void test8()
{
	unsigned long t1,t2;
	t1 = (unsigned long)GetCycleCount();
	foo();//dosomething
	t2 = (unsigned long)GetCycleCount();
```

```
        printf("Use Time:%f\n",(t2 - t1)*1.0/FREQUENCY); //FREQUENCY  CPU 的频率
}
```

总结，方法 1,2,7,8 可以在 linux 环境下执行，方法 1,2,3,4,5,6 可以在 windows 环境下执行.其中，timeGetTime()和 GetTickCount()的返回值类型为 DWORD，当统计的毫妙数过大时，将会使结果归 0，影响统计结果.

测试结果，windows 环境下，主频为 1.6GHz，单位为秒.

1 Use Time:0

2 Use Time:0.390000

3 Use Time:0.388000

4 Use Time:0.394704

5 Use Time:0.407000

6 Use Time:0.398684